GROUP ASSIGNMENT – 2

Submission Guidelines

- This is a **group assignment**.
- Submit a single ZIP file containing:
  - Source Code
  - Screenshots of outputs and visualizations
- Clearly mention the names and roll numbers of all group members.

PROBLEM 1: SMART HOME ENERGY CONSUMPTION ANALYSIS USING BAYESIAN NETWORKS

Background

A smart-home company is developing an intelligent energy management system capable of predicting whether a household is likely to experience **high energy consumption** on a given day, and of explaining **why** a high-consumption day occurred.

Energy usage depends on uncertain factors such as weather conditions, the type of day (weekday or weekend), occupancy patterns, air-conditioner usage, and appliance load. Since these relationships are probabilistic in nature, the company has decided to model the problem using a **Bayesian Network**.

The company currently has an expert-designed Bayesian Network based on domain knowledge. They would also like to investigate how well a similar model can be learned automatically from data, and how the amount of data and the choice of estimator affect the learned model.

Your task is to construct the Bayesian Network, perform predictive and diagnostic inference, analyse conditional independencies, generate synthetic data, learn network parameters from the generated data, and compare the expert-defined and learned models.

Variables

Weather (W)

Represents daily weather conditions.

Possible Values

- Hot
- Mild
- Cold

Day Type (D)

Represents whether the day is a working day or a holiday/weekend.

Possible Values

- Weekday
- Weekend

Occupancy (O)

Represents the number of people actively using the house.

Possible Values

- High
- Low

Depends on: Day Type

Air Conditioner Usage (A)

Represents whether the air conditioner is used heavily.

Possible Values

- On
- Off

Depends on:

- Weather
- Occupancy

Appliance Load (L)

Represents usage of household electrical appliances.

Possible Values

- Heavy
- Light

Depends on: Occupancy

Energy Consumption (E)

Represents overall household energy consumption.

Possible Values

- High
- Low

Depends on:

- Air Conditioner Usage
- Appliance Load

Probability Tables

Prior Probabilities

Variable	Probability
P(Weather = Hot)	0.50
P(Weather = Mild)	0.30
P(Weather = Cold)	0.20
P(DayType = Weekday)	0.70
P(DayType = Weekend)	0.30

Occupancy

Day Type	P(Occupancy = High)
Weekday	0.45
Weekend	0.80

AC Usage

Weather	Occupancy	P(AC = On)
Hot	High	0.90
Hot	Low	0.60
Mild	High	0.40
Mild	Low	0.15
Cold	High	0.10
Cold	Low	0.05

Appliance Load

Occupancy	P(Heavy Load)
High	0.75
Low	0.35

Energy Consumption

AC Usage	Appliance Load	P(High Consumption)
On	Heavy	0.92
On	Light	0.68
Off	Heavy	0.55
Off	Light	0.12

The complementary probabilities (e.g. P(AC = Off), P(Light Load), P(Low Consumption)) are obtained as 1 minus the given value.

Tasks

Part A: Expert Bayesian Network

1. Construct the Bayesian Network using Python.
2. Define all Conditional Probability Tables (CPTs) and programmatically verify that every CPT distributions must sum to 1 for every parent configuration.
3. Visualize the Bayesian Network.
4. Compute the following probabilities:
    - Q1: P(EnergyConsumption = High)
    - Q2: P(EnergyConsumption = High | Weather = Hot)
    - Q3: P(EnergyConsumption = High | DayType = Weekend)
    - Q4: P(EnergyConsumption = High | Weather = Cold, Occupancy = High)
    - Q5: P(Weather = Hot | EnergyConsumption = High) *(diagnostic query)*
    - Q6: P(AC = On | EnergyConsumption = High)
    - Q7: P(AC = On | EnergyConsumption = High, ApplianceLoad = Heavy)
5. Compare Q6 and Q7 and explain the observed change using the concept of **explaining away**.
6. Calculate Q1 **by hand** (show the summation steps) and confirm that it matches the value produced by your code.

Part B: Conditional Independence Analysis

Using d-separation, state whether each of the following independence statements holds in the network, and justify your answer by identifying the relevant paths:

1. Weather ⊥ DayType
2. Weather ⊥ DayType | EnergyConsumption
3. Weather ⊥ ApplianceLoad | Occupancy

Verify each answer numerically by comparing the appropriate conditional probabilities obtained through inference (for example, compare P(Weather = Hot | E = High) with P(Weather = Hot | E = High, DayType = Weekend)).

Part C: Synthetic Data Generation

Generate synthetic datasets using ancestral sampling from the Bayesian Network. Implement the sampling procedure yourself by visiting the nodes in topological order; fix and report the random seed so that your results are reproducible.

Create datasets of size:

- 500 samples
- 5000 samples
- 20000 samples

For each dataset, report the empirical frequency of each Weather value and the number of samples observed for each (Weather, Occupancy) combination.

Part D: Learning from Data

For each generated dataset:

1. Learn the network parameters using **both**:
    - Maximum Likelihood Estimation (MLE), and
    - Bayesian Estimation (BDeu prior with equivalent sample size = 10)
2. Construct the two learned Bayesian Networks (using the same structure as the expert network).
3. Display the learned CPT for AC Usage alongside the expert CPT.
4. Perform the same inference queries (Q1 – Q7).

Part E: Comparative Analysis

Compare the results obtained from:

- Expert Model
- Learned Model (MLE)
- Learned Model (Bayesian Estimation)

Use the following format:

Query	Expert Model	MLE Model	Bayesian Model	|Diff| MLE	|Diff| Bayesian
Q1					
Q2					
Q3					
Q4					
Q5					
Q6					
Q7					

Repeat the comparison for all dataset sizes. Then:

1. Plot the mean absolute difference (over Q1 – Q7) against dataset size for both estimators.

2. Identify which query shows the largest error for the 500-sample dataset and explain why (hint: consider how often its evidence occurs in the data).
3. Discuss in a short paragraph which estimator performs better for small datasets and why.

PROBLEM 2: MARS ROVER SAMPLE COLLECTION PLANNING USING PDDL

Background

A space agency has deployed an autonomous rover on Mars to collect geological samples from various exploration sites and transport them to a central research base.

Due to communication delays, the rover must operate autonomously using plans generated by an automated planning system. The rover has two storage bays, some of the rocks are hard and can only be extracted with a drill, and one of the routes is a steep slope that can be travelled in only one direction.

Your task is to model the problem using PDDL, generate valid plans using a planner, and compare the plans produced by different search strategies.

Locations

- Base
- SiteA
- SiteB
- SiteC
- SiteD
- SiteE
- SiteF

Samples

Sample	Type
Rock1	Soft
Rock2	Soft
Rock3	Hard
Rock4	Soft
Rock5	Hard

Soft rocks can be collected directly. Hard rocks can be collected only if the rover is carrying the drill.

Rover and Equipment

- Rover: **Curiosity**, with two storage bays: **Bay1** and **Bay2**
- Equipment: one drill (**Drill1**)

Connectivity

The rover may move only between connected locations.

Two-way routes

- Base ↔ SiteA
- Base ↔ SiteB
- SiteA ↔ SiteC
- SiteB ↔ SiteD
- SiteC ↔ SiteE
- SiteD ↔ SiteE
- SiteE ↔ SiteF

One-way route

- SiteC → SiteD (steep downhill slope; the rover cannot travel from SiteD to SiteC directly)

Initial State

Object	Location
Rover	Base
Drill1	SiteE
Rock1	SiteA
Rock2	SiteB
Rock3	SiteC
Rock4	SiteD
Rock5	SiteF

Both storage bays are initially empty and the rover is not carrying the drill.

Goal State

All rocks must be delivered to Base, both storage bays must be empty,
and the rover must finish the mission at Base.

Available Actions

Move

Move between connected locations (respecting the direction of the one-way route).

Pick Drill

Pick up the drill at the current location. Once picked up, the rover keeps the drill for the rest of the mission.

Collect

Pick up a rock from the current location into an empty storage bay. Hard rocks require the drill.

Deposit

Drop a rock from a storage bay at the current location, freeing that bay.

Constraints

1. Each storage bay can hold only one rock, so the rover can carry at most two rocks at a time.
2. Rover and rock must be at the same location before collection.
3. Hard rocks can be collected only after the rover has picked up the drill.
4. Depositing can occur only at the rover's current location.
5. Movement is allowed only through connected locations; SiteC → SiteD can be used in one direction only.
6. The rover must end the mission at Base.

Tasks

Task 1: Domain Modeling

Create `Domain.pddl` containing:

- Types
- Predicates
- Actions

Use only the :strips and :typing requirements (no numeric fluents or negative preconditions).

Task 2: Problem Modeling

Create `Problem.pddl` containing:

- Objects
- Initial State
- Goal State

Task 3: Plan Generation

Generate valid plans using a planning system with **two** different search configurations:

- a satisficing configuration (e.g. greedy best-first search), and
- an optimal configuration (e.g. A* with an admissible heuristic, or breadth-first search).

For each configuration, record:

- Sequence of actions
- Plan length
- Total number of actions
- Number of states expanded and planning time

Task 4: Goal Verification

Verify that the generated plan satisfies the specified goal state by simulating the plan step by step (or by using a plan validator such as VAL). Show the final state and confirm that all rocks are at Base, both bays are empty, and the rover is at Base.

Task 5: Extended Scenario

Add:

- Rock6 (Soft) at SiteE
- Rock7 (Hard) at SiteF

Change:

- A dust storm closes the route Base ↔ SiteB in both directions.

Goal:

- Rock1 – Rock7 at Base
- Rover at Base

Generate a new plan (create `Problem_extended.pddl`).

Task 6: Plan Analysis

1. Compare the plan lengths of the satisficing and optimal configurations for both scenarios and explain the difference.
2. Explain how the optimal plan uses the two storage bays and when the drill is picked up.
3. In the extended scenario, explain how the rover reaches SiteB and SiteD after the dust storm, and whether the one-way route is used.

Deliverables

- Domain.pddl
- Problem.pddl
- Problem_extended.pddl
- Planner output (both configurations, both scenarios)
- Analysis report

PROBLEM 3: ROYAL ARCHIVE ACCESS VERIFICATION USING RESOLUTION REFUTATION

Background

A royal archive stores sealed historical records. Before a person can be trusted with an archive record, the person's role and actions must satisfy a sequence of logical rules. The archive team wants to use **First-Order Predicate Logic (FOL)** and **Prolog** to verify whether a particular person should be considered trusted.

Your task is to represent the knowledge as logical rules, convert the formulas into clause form, and use **Resolution Refutation** to prove the required conclusion.

Facts and Rules

1. Every archivist who verifies a record is trusted.
2. Every trusted person who has access to a sealed record is permitted to inspect that record. (Being permitted to inspect is different from actually inspecting.)
3. Anyone who inspects a record and finds a hidden mark in it verifies that record.
4. Every record stored in the royal archive is sealed.
5. scroll_7 is stored in the royal archive.
6. Mira has access to scroll_7.
7. Mira inspects scroll_7 and finds a hidden mark in it.
8. Mira is an archivist.

Goal

Prove that: **Mira is trusted.**

Use the Resolution Refutation method in First-Order Predicate Logic (FOL).

Tasks

Task 1: Knowledge Representation

Represent all facts and rules in First-Order Predicate Logic. Clearly identify predicates, constants, and variables.

Task 2: Negate the Goal

Add the negation of the goal to the knowledge base for resolution refutation.

Task 3: Convert to Clause Form

Remove implications, move negations inward, standardize variables apart, perform Skolemization where necessary, and convert all formulas into CNF clauses.

Task 4: Prolog Implementation

Create a Prolog program that records each transformation stage in separate sections or predicates.

Task 5: Resolution Refutation

Implement or use a resolution procedure to show each resolvent in order. Continue until the empty clause ([]) is derived.

Task 6: Proof Verification

Run the program and verify that the empty clause is produced, proving the goal follows from the facts and rules.

Expected Prolog File Structure

```
% Step 1: Predicate logic representation
% Step 2: Negation of goal
% Step 3: Implication removal
% Step 4: Negation movement
% Step 5: Standardization
% Step 6: Skolemization
% Step 7: CNF clauses
% Step 8: Resolution implementation and execution
```

Deliverables

1. A single Prolog file (**royal_archive_access.pl**) containing:
    - The code and comments for all transformation steps.
    - The resolution refutation process showing the derivation of the empty clause.

2. A screenshot of the final proof output showing the empty clause or contradiction.

Challenge

The proof requires combining multiple rules to derive intermediate predicates before reaching the goal. Do not assume the conclusion directly; show the chain of reasoning through resolution. Note that not every rule given is needed for the proof.